

# CSP Redundanzmodus - Einrichtungshandbuch

---

## 1. Übersicht

---

Der CSP-Redundanzmodus ermöglicht den Betrieb von zwei CSP-Instanzen (Cell Service Providern) für dieselbe Zelle. Ziel ist die unterbrechungsfreie Weiterführung des Betriebs bei Ausfall einer Instanz - auch während laufender Rezepte.

### Konzept

- **Primary CSP:** Aktive Instanz. Liest und schreibt OPC-UA Items, steuert PLC-Kommunikation, führt Rezepte und Workflows aus. Nur der Primary darf auf die Datenbank schreiben und Auditeinträge bzw. Messages erzeugen.
- **Secondary CSP:** Passive Instanz (Standby). Verhält sich wie ein RSP - hört auf MQTT-Daten mit, kann Workflows nur über MQTT-Commands triggern.

Beide Instanzen verwenden dieselbe Cell-ID. Der **ALTERNATIVE-Knoten** verwendet zusätzlich einen **statischen Offset von 90.000** auf der MQTT-Topic-Ebene — unabhängig davon, ob er gerade Primary oder Secondary ist. Dadurch können beide CSPs gleichzeitig am selben MQTT-Broker und an derselben Datenbank arbeiten, ohne sich gegenseitig zu stören.

### Rollenverteilung

| Eigenschaft                       | Primary | Secondary                        |
|-----------------------------------|---------|----------------------------------|
| OPC-UA Schreibzugriff auf SPS     | Ja      | Ja                               |
| Datenbank-Schreibzugriff          | Ja      | Ja                               |
| Auditeinträge / Messages erzeugen | Ja      | Nein                             |
| Rezeptausführung                  | Aktiv   | Passiv (übernimmt bei Failover)  |
| MQTT-Daten publizieren            | Ja      | Ja                               |
| MQTT-Daten empfangen              | Ja      | Ja (hört auf Primary-Topics mit) |

| Eigenschaft        | Primary | Secondary                   |
|--------------------|---------|-----------------------------|
| Failover-Übernahme | -       | Automatisch nach 5 Sekunden |

## Knotentypen (Node Types)

Es gibt zwei Knotentypen innerhalb eines CSP-Clusters:

- **ORIGINAL** ( `cluster.nodeType=1` ): Die "bevorzugte" Instanz publiziert unter der originalen Cell-ID. Bei gleichzeitigem Start beider CSPs erhält der ORIGINAL-Knoten Vorrang bei der Primary-Wahl.
- **ALTERNATIVE** ( `cluster.nodeType=2` ): Die zweite Instanz publiziert unter der alternativen Cell-ID (Cell-ID + 90000 Offset). Weicht dem ORIGINAL-Knoten, wenn beide gleichzeitig Primary beanspruchen.

**Wichtig:** Der Knotentyp bestimmt nur die Priorität bei der Wahl. Beide Knoten können sowohl Primary als auch Secondary werden.

## 2. Voraussetzungen

### Hardware

- Zwei separate Rechner (oder VMs), auf denen jeweils eine CSP-Instanz läuft.
- Beide Rechner müssen Netzwerkzugang haben.

### Netzwerk

- Gemeinsamer **MQTT-Broker**, erreichbar von beiden CSP-Instanzen.
- Beide CSPs müssen die **SPS per OPC-UA** erreichen können.
- Beide CSPs brauchen Zugriff auf einen **OPC-UA Server für die Cluster-Kommunikation**. Details in Abschnitt 4.
- Beide CSPs brauchen Zugriff auf den Datenbank-Cluster.

## SPS-seitige Vorbereitung

Auf der SPS müssen **4 OPC-UA Items** für die Cluster-Kommunikation angelegt werden. Details in Abschnitt 4.

---

## 3. Konfiguration der vis.properties

---

Die `userdata/vis.properties` Datei muss auf **beiden** CSP-Instanzen konfiguriert werden.

### 3.1 Gemeinsame Einstellungen (beide CSPs identisch)

Diese Properties müssen auf beiden Instanzen **gleich** sein:

```
# Cell-ID der Zelle (identisch auf beiden CSPs)
cellbroker.cellId

# MQTT-Broker URL (gleicher Broker für beide)
cellbroker.mqttBrokerUrl

# Erlaubte RSP-Clients
cellbroker.clientNames

# OPC-UA Verbindung zur SPS (gleiche SPS)
OPC.SimaticNet.ipAddress
OPC.SimaticNet.namespaceUri
OPC.SimaticNet.simulated

# Datenbank (gleiche CrateDB-Instanz)
jdbc.ip
jdbc.port
```

### 3.2 Instanzspezifische Einstellungen

Diese Properties **unterscheiden sich** zwischen den zwei CSP-Instanzen:

## CSP-Instanz "Original" (bevorzugter Primary)

```
# Knotentyp: 1 = ORIGINAL (hat Vorrang bei der Primary-Wahl)
cluster.nodeType=1

# Eindeutiger Maschinenname zur Identifikation
z.B: hmi.machineName=CSP 001 Original
```

## CSP-Instanz "Alternative" (Standby-Knoten)

```
# Knotentyp: 2 = ALTERNATIVE (weicht dem Original bei Gleichzeitigkeit)
cluster.nodeType=2

# Eindeutiger Maschinenname zur Identifikation
z.B: hmi.machineName=CSP 001 Alternative
```

## 3.3 Cluster-Kommunikation (ClusterInfo-Items)

Die Cluster-Kommunikation läuft über 4 OPC-UA Items, die auf einem von beiden CSPs erreichbaren OPC-UA Server liegen. Die Item-Pfade werden standardmässig über `cluster.*` Properties in der `vis.properties` konfiguriert. Alternativ kann für Testzwecke eine separate `fspClusterInfo`-Verbindung verwendet werden.

### Item-Pfade für Cluster-Items (Standard)

Die Cluster-Items müssen auf einem OPC-UA Server bereitgestellt werden. Die Pfade werden in der `vis.properties` konfiguriert und verweisen auf Items im AutomationSphere Item-Manager — unabhängig davon, auf welcher Connection sie definiert sind.

```
# Item-Pfade für Cluster-Items
cluster.original.timestamp=OPC.SimaticNet.Cluster.original.timestamp
cluster.original.payload=OPC.SimaticNet.Cluster.original.payload
cluster.alternative.timestamp=OPC.SimaticNet.Cluster.alternative.timestamp
cluster.alternative.payload=OPC.SimaticNet.Cluster.alternative.payload
```

## Alternativ: fspClusterInfo-Verbindung (Testzwecke)

Wenn die `cluster.*` Properties **nicht** gesetzt sind, werden die Default-Pfade auf der `fspClusterInfo`-Verbindung verwendet. Diese Variante eignet sich für Testzwecke und erfordert eine separate OPC-UA Verbindung:

- `Cluster.original.timestamp`
- `Cluster.original.payload`
- `Cluster.alternative.timestamp`
- `Cluster.alternative.payload`

```
# IP-Adresse und Port des OPC-UA Servers für die Cluster-Kommunikation
# Dies ist der Server, auf dem die 4 Cluster-Items liegen (s. Abschnitt 4)
fspClusterInfo.ipAddress=10.80.6.200:4840
```

**Hinweis:** Wenn `fspClusterInfo.ipAddress` **nicht** gesetzt ist, läuft die `fspClusterInfo`-Verbindung im simulierten Modus. Sind weder die `cluster.*` Properties noch `fspClusterInfo.ipAddress` gesetzt, ist der Redundanzmodus **nicht** funktionsfähig.

## 3.4 Vollständige Property-Referenz

| Property                                   | Typ         | Pflicht | Beispiel                                   | Beschreibung                                      |
|--------------------------------------------|-------------|---------|--------------------------------------------|---------------------------------------------------|
| <code>cluster.nodeType</code>              | Integer     | Ja      | <code>1</code> oder <code>2</code>         | <code>1</code> = ORIGINAL<br>ALTERNATIVE          |
| <code>cluster.original.timestamp</code>    | Long        | Nein    | <code>Cluster.original.timestamp</code>    | Item-Pfad<br>Timestamp<br><code>Cluster.or</code> |
| <code>cluster.original.payload</code>      | String(200) | Nein    | <code>Cluster.original.payload</code>      | Item-Pfad<br>(Default:<br><code>Cluster.or</code> |
| <code>cluster.alternative.timestamp</code> | Long        | Nein    | <code>Cluster.alternative.timestamp</code> | Item-Pfad<br>Timestamp<br><code>Cluster.al</code> |
| <code>cluster.alternative.payload</code>   | String(200) | Nein    | <code>Cluster.alternative.payload</code>   |                                                   |

| Property                              | Typ     | Pflicht   | Beispiel                 | Beschreibung                                                    |
|---------------------------------------|---------|-----------|--------------------------|-----------------------------------------------------------------|
|                                       |         |           |                          | Item-Pfad<br>Payload (D<br>Cluster.al                           |
| <code>fspClusterInfo.ipAddress</code> | String  | Nein*     | 10.80.6.200:4840         | OPC-UA Se<br>Heartbeat<br>wenn <code>clus</code><br>nicht geset |
| <code>cellbroker.cellId</code>        | Integer | Ja        | 815                      | Cell-ID (ide<br>CSPs)                                           |
| <code>cellbroker.mqttBrokerUrl</code> | String  | Ja        | tcp://192.168.1.100:1883 | MQTT-Bro                                                        |
| <code>hmi.machineName</code>          | String  | Empfohlen | CSP 815 Original         | Anzeigena<br>Unterschei                                         |

## 4. OPC-UA Items auf der SPS (Cluster-Kommunikation)

### 4.1 Überblick

Die beiden CSP-Instanzen kommunizieren ihren Status (Primary/Secondary) und Heartbeat-Signale über **4 OPC-UA Items**. Diese Items müssen auf einem OPC-UA Server liegen, der von beiden CSPs erreichbar ist.

Die Items sind paarweise aufgebaut — je ein Paar für den ORIGINAL-Knoten und eines für den ALTERNATIVE-Knoten:

| Item-Identifizier                       | OPC-UA Datentyp | Zugriff        | Beschreibung                                                          |
|-----------------------------------------|-----------------|----------------|-----------------------------------------------------------------------|
| <code>Cluster.original.timestamp</code> | UInt64          | Read/<br>Write | Heartbeat-Zeitstempel des ORIGINAL-Knotens (Millisekunden seit Epoch) |
| <code>Cluster.original.payload</code>   | String(200)     | Read/<br>Write | JSON-Payload des ORIGINAL-Knotens (Modus und Kommando, siehe 4.2)     |

| Item-Identifizier                          | OPC-UA Datentyp | Zugriff        | Beschreibung                                                             |
|--------------------------------------------|-----------------|----------------|--------------------------------------------------------------------------|
| <code>Cluster.alternative.timestamp</code> | UInt64          | Read/<br>Write | Heartbeat-Zeitstempel des ALTERNATIVE-Knotens (Millisekunden seit Epoch) |
| <code>Cluster.alternative.payload</code>   | String(200)     | Read/<br>Write | JSON-Payload des ALTERNATIVE-Knotens (Modus und Kommando, siehe 4.2)     |

## 4.2 Payload-Format (JSON)

Das Payload-Item enthält einen JSON-String mit folgendem Aufbau:

```
{
  "mode": "PRIMARY",
  "command": "REQUEST_PRIMARY_STATE"
}
```

### Mögliche Werte für `mode` :

| Wert                                    | Bedeutung                     |
|-----------------------------------------|-------------------------------|
| <code>PRIMARY</code>                    | Instanz ist aktiver Primary   |
| <code>SECONDARY</code>                  | Instanz ist Standby           |
| <code>REQUEST_BECOMING_PRIMARY</code>   | Instanz möchte Primary werden |
| <code>TRANSITION_TO_PRIMARY</code>      | Übergang zu Primary läuft     |
| <code>REQUEST_BECOMING_SECONDARY</code> | Instanz wechselt zu Secondary |
| <code>TRANSITION_TO_SECONDARY</code>    | Übergang zu Secondary läuft   |

### Mögliche Werte für `command` :

| Wert                               | Bedeutung                         |
|------------------------------------|-----------------------------------|
| <code>REQUEST_PRIMARY_STATE</code> | Anfrage: Darf ich Primary werden? |
|                                    | Ablehnung: Nein, ich bin Primary  |

| Wert                | Bedeutung                |
|---------------------|--------------------------|
| DENY_PRIMARY_STATE  |                          |
| GRANT_PRIMARY_STATE | Zustimmung: Ja, übernimm |

## 4.3 Heartbeat-Mechanismus

- Jede CSP-Instanz schreibt **jede Sekunde** ihren aktuellen Zeitstempel (Millisekunden seit Epoch) in ihr eigenes `timestamp`-Item.
- Die andere Instanz liest diesen Zeitstempel und erkennt anhand eines **5-Sekunden-Schwellwerts**, ob die Gegenstelle noch lebt.
- Bleibt der Heartbeat einer Instanz mehr als 5 Sekunden aus, wird ein Failover eingeleitet.

## 5. clusterSyncItems - Synchronisierte Items

### 5.1 Zweck

Die `clusterSyncItems`-Gruppe definiert, welche Items zwischen Primary und Secondary synchronisiert werden. Wenn der Secondary zum Primary wird (Failover), übernimmt er die aktuellen Werte dieser Items vom bisherigen Primary.

### 5.2 Standard-Items

Folgende Items werden standardmäßig in der `asphere.xml` synchronisiert:

#### Batch- und Operations-Items:

| Item-Name                              | Beschreibung              |
|----------------------------------------|---------------------------|
| <code>hmi.batch.batchId</code>         | Aktuelle Batch-ID         |
| <code>hmi.batch.phaseId</code>         | Aktuelle Phasen-ID        |
| <code>hmi.operation.operationId</code> | Aktuelle Operations-ID    |
| <code>hmi.operation.recordingId</code> | Aktuelle Aufzeichnungs-ID |

**Timer-Items** (werden automatisch synchronisiert, Details in Abschnitt 7.4):



| Item-Gruppe    | Items                                        | Beschreibung               |
|----------------|----------------------------------------------|----------------------------|
| Start-Booleans | <code>csp.timer.start1/2/3</code>            | Timer-Laufstatus           |
| Stop-Booleans  | <code>csp.timer.stop1/2/3</code>             | Timer-Stopp-Status         |
| Reset-Booleans | <code>csp.timer.reset1/2/3</code>            | Timer-Reset-Status         |
| Elapsed Millis | <code>csp.timer.timeMilliseconds1/2/3</code> | Verstrichene Millisekunden |
| Namen          | <code>csp.timer.name1/2/3</code>             | Timer-Bezeichnungen        |

### 5.2.1 Programmatisch registrierte Items

Zusaetzlich zu den in der `asphere.xml` deklarierten Items werden weitere Items zur Laufzeit programmatisch zur `clusterSyncItems`-Gruppe hinzugefuegt. Diese muessen **nicht** manuell konfiguriert werden:

**Profil-Items** ( `ProfileHandler` ): Fuer jedes konfigurierte Profil (z.B. Sensoren, Aktoren, Ventile) werden drei Items automatisch registriert:

| Item                                                                             | Beschreibung                 |
|----------------------------------------------------------------------------------|------------------------------|
| <code>*.Profil.SP_RExecTime</code> / <code>*.Profil.SP-EU_RExecTime</code>       | Ausfuehrungszeit des Profils |
| <code>*.Profil.SP_RState</code> / <code>*.Profil.SP-EU_RState</code>             | Profilstatus                 |
| <code>*.Profil.SP_RProfileName</code> / <code>*.Profil.SP-EU_RProfileName</code> | Aktiver Profilname           |

**Hinweis:** Die Anzahl der Profil-Items haengt von der Anlagenkonfiguration ab. Bei einer typischen Zelle mit 10 Profilen werden 30 zusaetzliche Items synchronisiert.

## 5.3 Synchronisations-Mechanismus

- Im **Primary-Modus**: Lokale Items (NullItems) sind aktiv und werden lokal beschrieben. Keine Synchronisation noetig.
- Im **Secondary-Modus**: Die lokalen NullItems werden mit den entsprechenden Remote-Items (BridgelItems) des Primary verknuepft (Master-Binding). Aenderungen des Primary werden automatisch ueber MQTT uebernommen.
- Bei **Failover** (Before-Disconnect-Pattern):

- Der `ClusterItemSynchronizationHandler` erkennt den Wechsel zu PRIMARY.
- **Vor** dem Entfernen der Master-Bindings werden alle registrierten `beforeMasterDisconnect`-Consumer aufgerufen. Diese koennen die aktuellen Werte aus den NullItems lesen, solange die Master-Bindings noch aktiv sind.
- Erst danach werden die Master-Bindings entfernt ( `setMasterItem(null)` ).
- Der neue Primary arbeitet mit den zuvor gesicherten Werten weiter.

**Technischer Hintergrund:** Das Before-Disconnect-Pattern ist notwendig, weil das Entfernen der Master-Bindings die NullItem-Werte auf ihre Defaults zuruecksetzt. Consumer wie der `AlternativeTimerController` muessen ihre Werte lesen, bevor das passiert.

## 5.4 Weitere Items hinzufuegen

Falls zusaezliche Items synchronisiert werden muessen, sollten diese in die `clusterSyncItems`-subGroup eingefuegt werden.

# 6. Inbetriebnahme

## 6.1 Checkliste vor dem Start

- [ ] OPC-UA Items auf der SPS/dem OPC-UA Server angelegt (4 Items, siehe Abschnitt 4)
- [ ] `vis.properties` auf CSP "Original" konfiguriert ( `cluster.nodeType=1` )
- [ ] `vis.properties` auf CSP "Alternative" konfiguriert ( `cluster.nodeType=2` )
- [ ] Cluster-Verbindung konfiguriert (eine der beiden Optionen):
- **Standard:** `cluster.original.*` und `cluster.alternative.*` Properties auf beiden CSPs gesetzt + Items auf dem jeweiligen OPC-UA Server vorhanden
- **Testzwecke:** `fspClusterInfo.ipAddress` auf beiden CSPs gesetzt (Fallback, wenn `cluster.*` Properties nicht gesetzt)
- [ ] `cellbroker.cellId` auf beiden CSPs identisch
- [ ] `cellbroker.mqttBrokerUrl` auf beiden CSPs identisch
- [ ] `asphere.xml` auf beiden CSPs identisch
- [ ] MQTT-Broker erreichbar von beiden CSPs
- [ ] OPC-UA Server (SPS) erreichbar von beiden CSPs

- [ ] OPC-UA Server (Cluster-Items) erreichbar von beiden CSPs (nur Variante A)
- [ ] CrateDB erreichbar von beiden CSPs
- [ ] `app.db.backupscheduler=true` auf **beiden** CSPs gesetzt (siehe Abschnitt 7.6)

## 6.2 Startreihenfolge

1. **MQTT-Broker starten** (falls nicht bereits laufend)
2. **CrateDB starten** (falls nicht bereits laufend)
3. **CSP "Original" starten** (`cluster.nodeType=1`)
4. Beim Start versucht diese Instanz, den Primary-Status zu erlangen.
5. Nach 5 Sekunden ohne Einspruch wird sie Primary.
6. **CSP "Alternative" starten** (`cluster.nodeType=2`)
7. Diese Instanz erkennt, dass der Original-Knoten bereits Primary ist.
8. Sie wird automatisch zum Secondary.

**Hinweis:** Die Startreihenfolge ist nicht zwingend. Auch bei gleichzeitigem Start wird der ORIGINAL-Knoten bevorzugt Primary. Der Mechanismus der Wahl ist deterministisch.

## 6.3 Verifikation nach dem Start

### Am Primary CSP prüfen:

- `hmi.machineName` wird im UI angezeigt (z.B. "CSP 815 Original")
- MQTT-Topic `STATE/CSP815/APP/appmode` enthält den Wert `PRIMARY`
- OPC-UA Heartbeat: `Cluster.CSP815.original.timestamp` wird jede Sekunde aktualisiert
- OPC-UA Payload: `Cluster.CSP815.original.payload` zeigt `"mode":"PRIMARY"`

### Am Secondary CSP prüfen:

- MQTT-Topic `STATE/CSP90815/APP/appmode` enthält den Wert `SECONDARY`
- OPC-UA Heartbeat: `Cluster.CSP815.alternative.timestamp` wird jede Sekunde aktualisiert
- OPC-UA Payload: `Cluster.CSP815.alternative.payload` zeigt `"mode":"SECONDARY"`

**Hinweis zum Offset:** Der ALTERNATIVE-Knoten publiziert seine MQTT-Topics **immer** mit Cell-ID + 90.000, unabhängig davon, ob er Primary oder Secondary ist. Für Cell-ID 815 wird also CSP90815 als Client-Name verwendet.

## 7. Failover-Verhalten

### 7.1 Automatischer Failover

Der Failover wird automatisch ausgelöst, wenn der Secondary erkennt, dass der Heartbeat des Primary **länger als 5 Sekunden** ausbleibt.

#### Ablauf:

1. Primary CSP faellt aus (Prozess-Absturz, Netzwerkausfall, etc.)
2. Der Heartbeat-Zeitstempel des Primary wird nicht mehr aktualisiert
3. Nach 5 Sekunden erkennt der Secondary den Ausfall
4. Der Secondary wechselt in den Modus `TRANSITION_TO_PRIMARY`
5. Der Secondary wird zum neuen Primary
6. `ClusterItemSynchronizationHandler` erkennt den Wechsel zu PRIMARY: a. **Before-Disconnect-Phase:** Registrierte Consumer (z.B. `AlternativeTimerController`) lesen die aktuellen Werte aus den NullItems, solange die Master-Bindings noch aktiv sind.  
b. **Disconnect-Phase:** Die Master-Bindings werden entfernt ( `setMasterItem(null)` ).
7. Laufende Rezepte und Operationen werden fortgesetzt.

#### 7.1.1 Selbstüberwachung des Primary

Der Primary überwacht nicht nur den Heartbeat des Secondary, sondern auch seinen **eigenen** Zeitstempel. In periodischen Abständen prüft er, ob sein zuletzt geschriebener Heartbeat noch aktuell ist. Stellt er fest, dass sein eigener Zeitstempel veraltet ist (z.B. durch einen temporären Netzwerkausfall zum OPC-UA Server), geht er davon aus, dass der Secondary in der Zwischenzeit die Primary-Rolle übernommen hat. In diesem Fall leitet der Primary einen **automatischen Shutdown** ein, um einen Zustand mit zwei gleichzeitigen Primaries zu verhindern. Nach einem Neustart (manuell oder durch den Windows-Dienst) kommt der ehemalige Primary als Secondary zurück.

## 7.2 Zeitlicher Ablauf

```
t=0s    Primary-Ausfall
t=1s    Heartbeat fehlt (1. Prüfung)
t=2s    Heartbeat fehlt (2. Prüfung)
t=3s    Heartbeat fehlt (3. Prüfung)
t=4s    Heartbeat fehlt (4. Prüfung)
t=5s    Schwellwert überschritten -> Failover startet
t=~6s   Secondary ist neuer Primary
```

## 7.3 Rezept-Failover (Hot Start)

Wenn ein Rezept laeuft und der Primary ausfaellt, fuehrt der neue Primary einen **Hot Start** durch:

- Der Secondary hat waehrend des Normalbetriebs den Rezeptzustand laufend ueber MQTT mitgelesen.
- Beim Wechsel zu Primary erkennt die Instanz, dass auf der Gegenseite ein Rezept aktiv war, lokal aber keines laeuft.
- Das Rezept wird automatisch ab dem letzten bekannten Zustand fortgesetzt — ein manueller Neustart ist nicht erforderlich.
- Die SPS-seitige Logik laeuft unabhaengig weiter; der neue Primary uebernimmt die Kommunikation.
- Batch-Operationen (Auditeintraege, Messwert-Aufzeichnung) werden nach dem Wechsel direkt in die Datenbank geschrieben statt ueber MQTT delegiert.

## 7.4 Timer-Verhalten bei Failover

Die drei SCADA-Timer werden ueber die `clusterSyncItems` -Gruppe laufend zwischen Primary und Secondary synchronisiert (Laufstatus, verstrichene Zeit, Timer-Name). Bei einem Failover rekonstruiert der neue Primary den internen Timer-Zustand automatisch aus den zuletzt synchronisierten Werten. Laufende Timer zaehlen nahtlos weiter; gestoppte Timer behalten ihre verstrichene Zeit. Die Timer-Anzeige im UI aktualisiert sich ohne manuellen Eingriff.

## 7.5 Profil-Verhalten bei Failover

Profilzustaende (Ausfuehrungszeit, Status, aktiver Profilname) werden ueber die `clusterSyncItems` -Gruppe synchronisiert und stehen dem neuen Primary nach einem

Failover sofort zur Verfügung. Ein erneutes Laden oder Aktivieren der Profile ist nicht nötig.

## 7.6 Snapshot-Verhalten bei Failover

Im **Secondary-Modus** werden manuelle Snapshot-Anfragen per MQTT-Command an den Primary delegiert — der Secondary erstellt keine Datenbank-Snapshots direkt. Nach einem Failover übernimmt der neue Primary diese Rolle. Danach laufen geplante Snapshots nur auf dem neuen Primary.

**Wichtig:** Die Property `app.db.backupscheduler` muss auf **beiden** CSP-Instanzen auf `true` gesetzt sein. Der Quartz-Scheduler wird nur auf Instanzen angelegt, bei denen diese Property aktiviert ist. Steht sie nur auf einem CSP auf `true`, werden nach einem Failover keine Snapshots mehr erstellt.

## 7.7 Rückkehr des Original-Knotens

### Szenario A: Neustart nach Absturz

Wenn der ausgefallene Original-Knoten neu startet:

1. Er beantragt den Primary-Status ( `REQUEST_PRIMARY_STATE` ).
2. Der Alternative-Knoten ist bereits gesunder Primary und **lehnt ab** ( `DENY_PRIMARY_STATE` ).
3. Der Original-Knoten akzeptiert die Ablehnung und wird **Secondary**.

Der ORIGINAL-Knoten hat **nur bei gleichzeitigem Start** beider Instanzen Vorrang. Wenn ein gesunder Primary bereits existiert, wird der zurückkehrende Knoten — unabhängig vom Knotentyp — immer Secondary.

### Szenario B: Temporärer Verbindungsverlust

Falls der Original-Knoten nicht abstürzt, sondern lediglich vorübergehend die Verbindung zum OPC-UA Server verliert:

1. Die Selbstüberwachung (siehe Abschnitt 7.1.1) erkennt den veralteten eigenen Zeitstempel.
2. Der Primary leitet einen **automatischen Shutdown** ein.

3. Nach einem Neustart (manuell oder durch den Windows-Dienst) kommt der Knoten als Secondary zurück.

---

## 7.8 RSP-Verhalten bei Failover

Verbundene RSP-Instanzen (SCADA-Clients) bemerken den Rollenwechsel durch eine Statusänderung auf dem MQTT-Topic `STATE/<cellId>/APP/appmode`. Für den Benutzer am RSP ändert sich jedoch **nichts** — die Bedienoberfläche bleibt funktionsfähig, da der neue Primary die MQTT-Kommunikation nahtlos übernimmt. Der RSP sendet Befehle weiterhin an dieselbe Cell-ID; diese werden vom jeweils aktiven Primary verarbeitet.

## 7.9 Einzelbetrieb ohne Redundanz

Der Einzelbetrieb ohne Redundanz kann erzielt werden, indem einfach nur eine einzige CSP-Instanz betrieben wird. In diesem Fall:

- Der CSP startet als ORIGINAL-Knoten und beantragt den Primary-Status.
- Nach 5 Sekunden ohne Antwort einer Gegenstelle wird er automatisch Primary.
- Der ClusterManager läuft im Hintergrund, hat aber keine Auswirkung auf den Betrieb.
- Kein Offset wird angewendet (nur der ALTERNATIVE-Knoten verwendet den 90.000-Offset).
- Alle Funktionen (Rezepte, Timer, Snapshots, Datenbank) arbeiten wie im Nicht-Redundanz-Betrieb.

### Minimale Konfiguration für Einzelbetrieb:

Es müssen **keine** cluster-spezifischen Properties gesetzt werden. Die folgenden Properties können weggelassen werden:

```
# NICHT nötig für Einzelbetrieb:
# cluster.nodeType           (Default: ORIGINAL)
# cluster.original.*        (nicht benötigt)
# cluster.alternative.*     (nicht benötigt)
# fspClusterInfo ipAddress  (nicht benötigt)
```